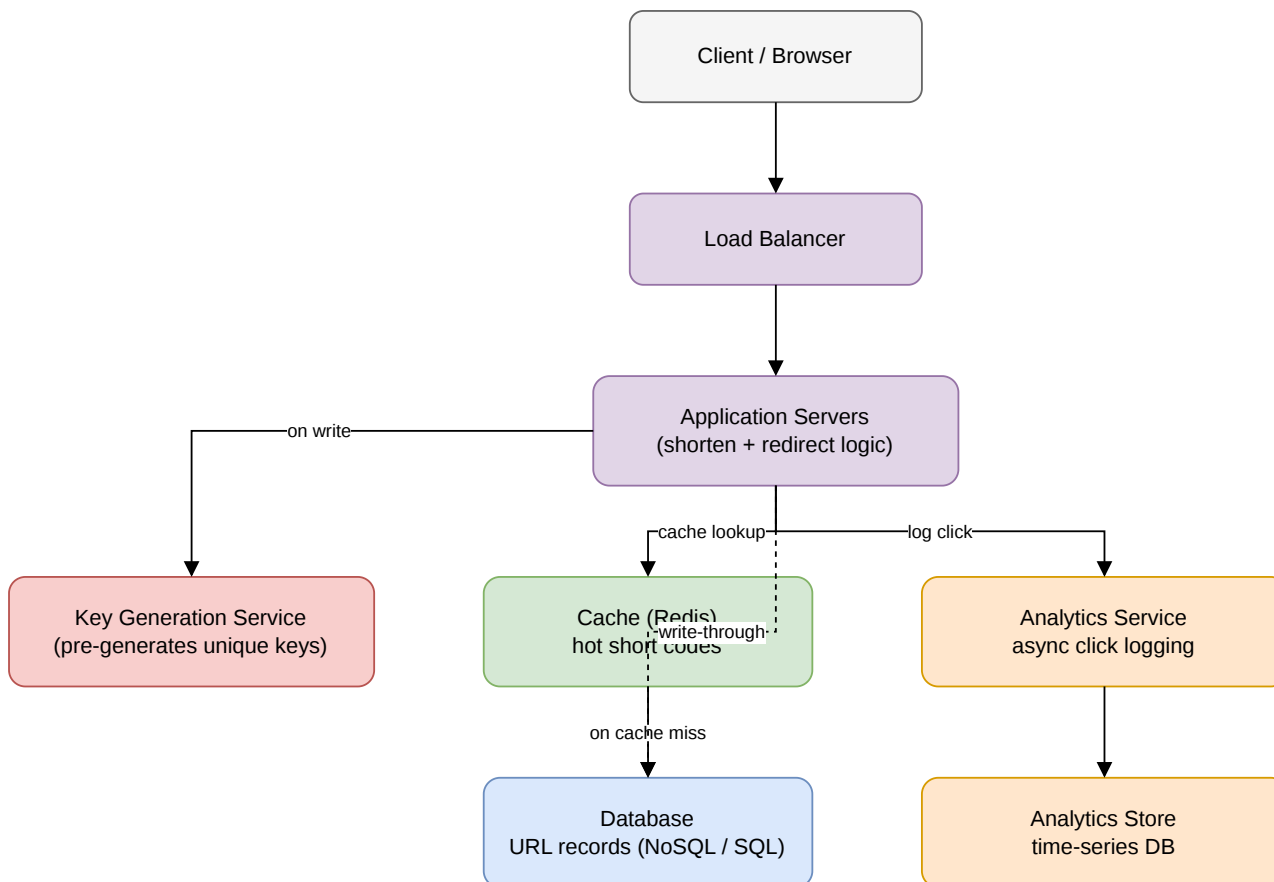




Read path: Client -> LB -> App -> Cache (hit) -> Redirect (301/302). DB hit only on cache miss.
Write path: Client -> LB -> App -> KGS (get key) -> DB write -> Cache warm.

TinyURL — Requirements

Functional Requirements

1. Given a long URL, generate a unique short URL
2. Redirect users from short URL to original long URL
3. Users can optionally choose a custom alias
4. Links can have an expiration date (default if unspecified)
5. (Optional) Analytics — click counts, timestamps, referrers
6. (Optional) User accounts to manage their own links

Non-Functional Requirements

- High availability — redirection is on the critical path
- Low latency — redirects in a few milliseconds
- High reliability — short URL always maps to same long URL
- Scalability — must handle heavy read traffic
- Security — short URLs should not be easily guessable
- Read:Write traffic is heavily skewed (~100:1)

Short Code Generation Strategy

Base62 [a-zA-Z0-9], 7 characters -> $62^7 \approx 3.5$ trillion combinations (comfortably covers billions of URLs).

Preferred approach: a Key Generation Service (KGS) pre-generates and stores random unique 7-char keys in a pool; app servers pull unused keys — avoids collision checks and predictability on the write path.

TinyURL — Back-of-the-Envelope Estimation

Assumptions

100M new URLs created per month | Read:Write ratio $\approx 100:1$ -> $\sim 10\text{B}$ redirects per month

Traffic

Writes/sec = $100\text{M} / (30 \times 24 \times 3600) \approx 40$ writes/sec
Reads/sec = $40 \times 100 \approx 4,000$ reads/sec

Storage

~ 500 bytes/record. Retain 5 years: $100\text{M/month} \times 60 \text{ months} = 6\text{B}$ records
Total storage $\approx 6\text{B} \times 500 \text{ bytes} \approx \sim 3 \text{ TB}$

Short Code Length

Base62 [a-zA-Z0-9] -> $62^7 \approx 3.5$ trillion combinations
7 characters is comfortably enough for 6B+ URLs

Bandwidth

Write: $40 \times 500 \text{ bytes/sec} \approx 20 \text{ KB/s}$ (negligible)
Read: $4,000 \times 500 \text{ bytes/sec} \approx 2 \text{ MB/s}$ (negligible)

Cache Sizing (80/20 rule)

20% of URLs generate 80% of traffic. Cache $\sim 20\%$ of daily reads: $(10\text{B/month} / 30) \times 0.2 \approx 67\text{M}$ records/day $\times 500 \text{ bytes} \approx \sim 35 \text{ GB}$ — fits comfortably in a Redis cluster.

TinyURL — API Design

POST /api/v1/shorten

Request body: { long_url, custom_alias (optional), expiry_date (optional) }
Response: { short_url: "https://tiny.url/abc1234" }
Creates a new short URL. Applies rate limiting to prevent abuse.

GET /{short_code}

Response: 301 (permanent, cacheable) or 302 (temporary, forces hit for analytics) redirect to long_url

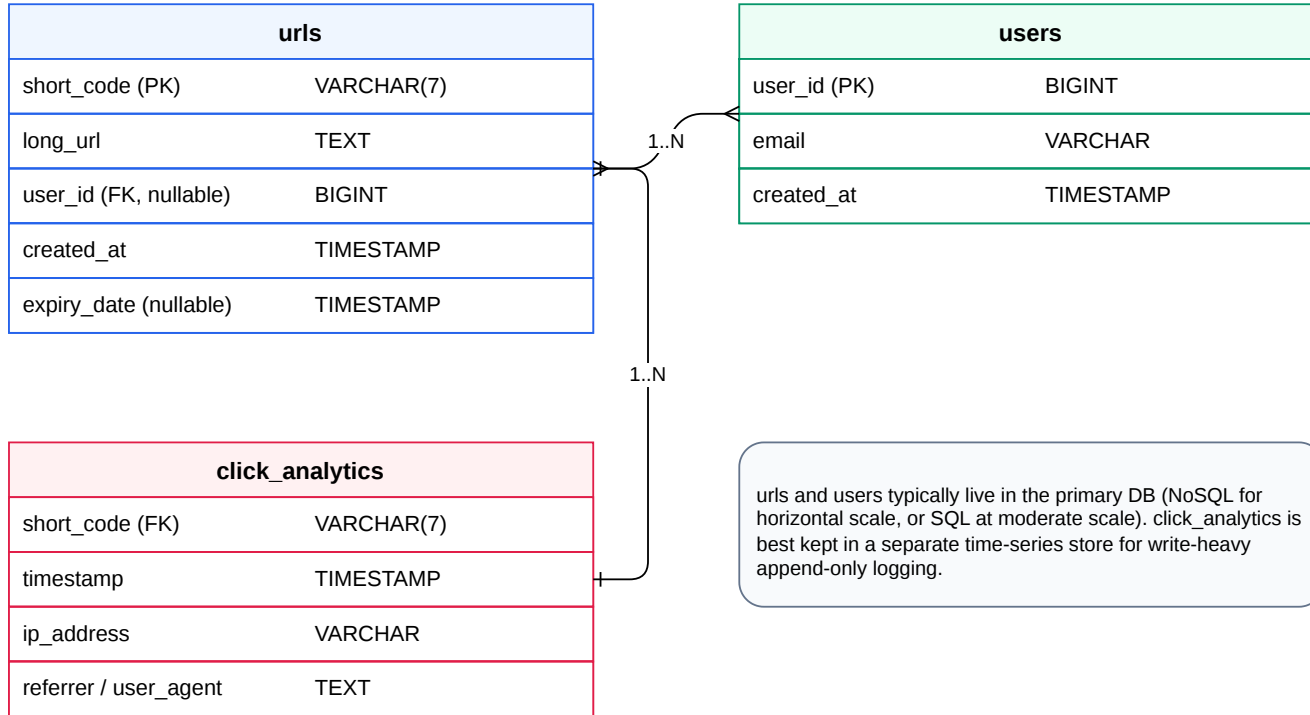
DELETE /api/v1/{short_code}

Response: 204 No Content. Deletes the mapping (owner/auth check applies).

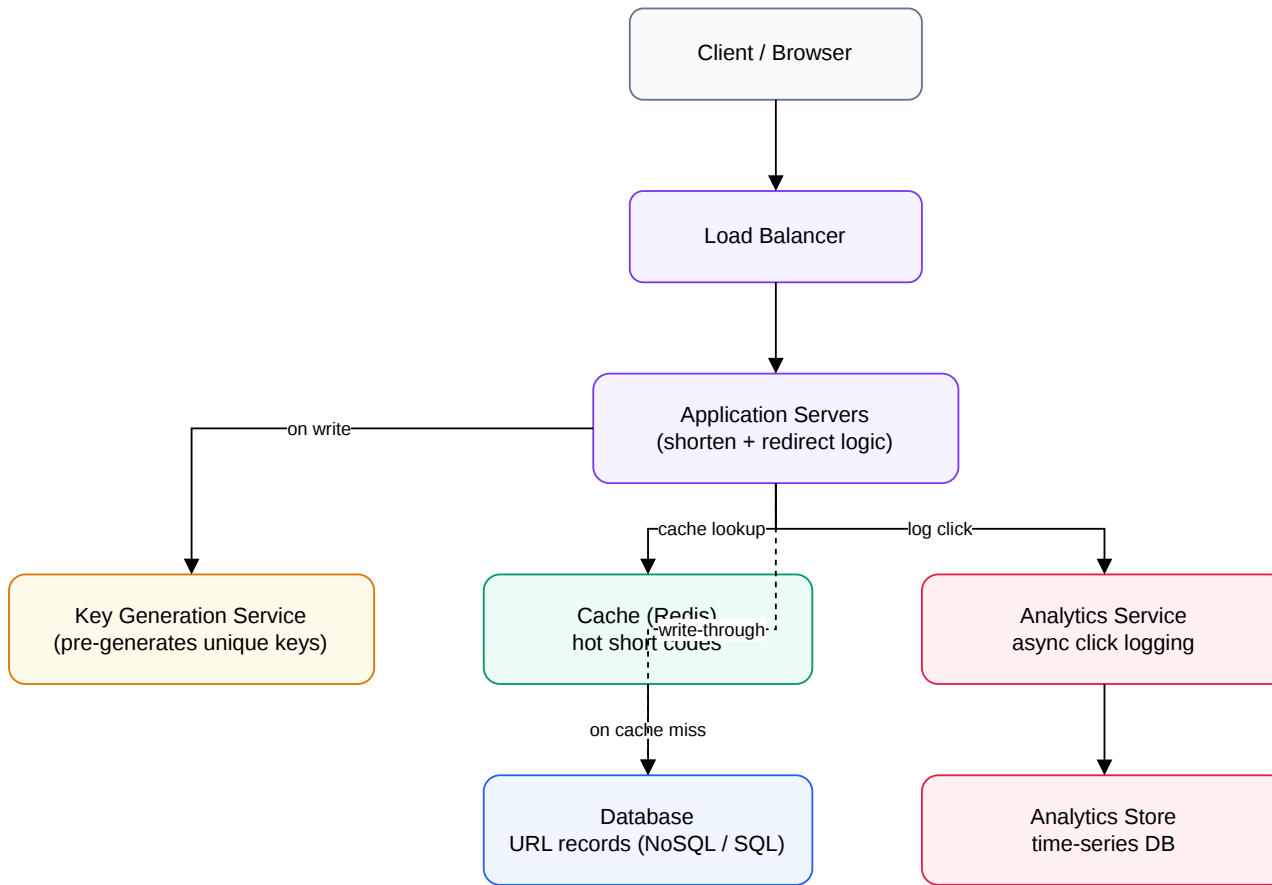
GET /api/v1/{short_code}/stats

Response: { clicks, created_at, last_accessed }

TinyURL — Database Schema



TinyURL — High Level Architecture



Read path: Client -> LB -> App -> Cache (hit) -> Redirect (301/302). DB hit only on cache miss.
Write path: Client -> LB -> App -> KGS (get key) -> DB write -> Cache warm.